

Eightfold Candidate Data Transformer - Technical Design Document

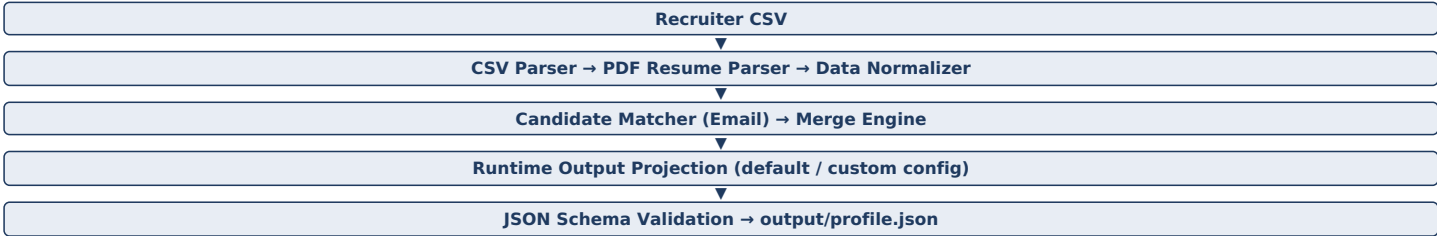
Eightfold Engineering Intern Assessment — Technical Design Document

Candidate: Kakkerla Swathi Priya **Email:** kakkerlaswathi26@gmail.com **Assessment:** Eightfold Engineering Intern (Jul-Dec 2026)

1. Problem Statement

Candidate information collected from heterogeneous sources (recruiter databases, resumes) is often incomplete, duplicated, or inconsistent. This project builds a Candidate Data Transformer that converts structured and unstructured candidate data into a standardized JSON profile — parsing structured/unstructured input, normalizing fields, merging duplicates, preserving provenance, generating confidence scores, validating against JSON Schema, and producing configurable output without code changes.

2. System Architecture



Modular architecture — each component has a single responsibility.

3. Canonical Output Schema

Field	Description
candidate_id	Unique candidate identifier
full_name	Candidate full name
emails	List of candidate email addresses — <i>string[]</i>
phones	Phone numbers — <i>string[]</i> (E.164)
company / title	Current company / job title
skills	Canonical skills + confidence + source
provenance	Source info for each field
overall_confidence	Overall confidence score

4. Data Sources

Structured — Recruiter CSV

Name, Email, Phone, Company, Title

Unstructured — PDF Resume (PyPDF2)

Name, Email, Phone, Skills

5. Processing Pipeline

- Step 1 — CSV Parsing:** Recruiter data loaded via Pandas, converted into candidate dictionaries.
- Step 2 — Resume Parsing:** PDF resumes parsed with PyPDF2; regex extracts email & phone; skills matched against a canonical skill list.
- Step 3 — Normalization:** Phones → E.164 format; skills → canonical capitalization (e.g. "python" → "Python").
- Step 4 — Candidate Matching:** Records matched by email — unique and present in both sources.
- Step 5 — Merge Strategy:** Recruiter data → company, title, identity. Resume → skills, contact info. Duplicates removed; missing values supplemented. When duplicate values are found, normalized recruiter data is preferred for identity fields, while resume data enriches skills and contact information.
- Step 6 — Confidence & Provenance:** Each skill carries a confidence score and extraction source; every field's origin is recorded.
- Step 7 — Runtime Configuration:** Output format driven by config files (default.json, custom.json) — include/exclude fields, rename output, set missing-value behavior — no code changes needed.
- Step 8 — Validation:** Final profile validated against JSON Schema before export.

```
{ "name": "Python", "confidence": 0.95, "sources": ["resume"] }
```

6. Design Decisions

- Modular architecture separates parsing, normalization, merging, and validation into independent components.
- Email is used as the primary matching key because it is unique and available in both data sources.
- Runtime configuration allows output customization without modifying application logic.
- JSON Schema validation guarantees consistent output.

7. Edge Cases Handled

Missing email/phone · empty skills section · invalid PDF files · candidate without matching resume · missing optional fields. Invalid records do not terminate execution.

8. Technologies Used

- Pandas** — CSV parsing
- PyPDF2** — PDF parsing
- phonenumbers** — phone normalization
- jsonschema** — output validation
- Python 3.12, Regex, Git, GitHub** — core language & tooling

9. Future Enhancements

OCR for scanned resumes · NLP-based skill extraction · fuzzy candidate matching · REST API · database persistence · web-based UI · support GitHub and LinkedIn profile ingestion using public APIs.

10. Conclusion — The solution transforms candidate information from multiple heterogeneous sources into a standardized JSON representation. Its modular architecture, configurable output pipeline, normalization strategy, validation mechanism, and provenance tracking make it maintainable, extensible, and suitable for real-world recruitment data processing.